



Smoker Lung Cancer Stage Detection Model

ABSTRACT

Smoking is the biggest risk factor for developing lung cancer, which is still a serious health problem. For enhancing patient outcomes and directing suitable treatment,

early identification and precise staging of lung cancer in smokers are essential. This study presents an advanced machine learning method for the early detection and prediction of lung cancer using a comprehensive dataset of information on patients with lung cancer, including their age, gender, exposure to air pollution, alcohol use, dust allergy, occupational risks, genetic risk, chronic lung disease, balanced diet, obesity, smoking, and passive smoking, as well as their chest pain, coughing up blood, fatigue, weight loss, shortness of breath, wheezing, and swallowing difficulties. Additionally, the study investigates the significance of various characteristics and offers information on possible biomarkers for the early detection of lung cancer. In summary, this work reveals major risk factors for lung cancer and underlines the promise of cutting-edge machine-learning approaches in the prediction of the illness. These predictive models can be applied to clinical practice to help medical professionals make defensible choices about early diagnosis and individualized treatment plans, ultimately improving patient outcomes and significantly lowering lung cancer mortality rates.

This study reveals how machine learning might help smokers with early-stage lung cancer diagnosis. The created model has the ability to help doctors make wise choices and deliver prompt therapies for patients with different stages of lung cancer. Our approach may enhance patient outcomes and provide a better understanding of how lung cancer develops in smokers by aiding in early identification and management.

This result of prediction shows that the Gradient Boosting Classifier and Decision Tree Classifier outperformed other models in terms of all evaluation metrics.

The analysis has also identified the most important features for the prediction of lung cancer, which are Coughing of Blood, Dust Allergy, Passive Smoker, Occupational Hazards, Air Pollution, Chronic Lung Disease, Shortness of Breath, Dry Cough, Snoring, and Swallowing Difficulty. The patients are to be mindful of these which can be a sign of lung cancer and help in early detection.

Table of Contents

CHAPTER-1	3
INTRODUCTION	3
1.1 OVERVIEW	3
1.2 PROPOSED WORK	4
1.3 RESEARCH QUESTIONS	4
1.4 PROJECT REQUIREMENTS	4
1.5 PROJECT RISKS	4
CHAPTER-2	5
LITERATURE REVIEW	5
2.1 BACKGROUND STUDY	5
2.2 CLASSIFICATION MODELS	5
CHAPTER-3	8
METHODOLOGY	8
3.1 OVERVIEW	8
3.2 DEFINE THE PROBLEM	9
3.3 DATA COLLECTION	9
3.4 DATA PRE-PROCESSING	10
3.5 DATA DESCRIPTION	10
3.6 DATA MODELLING	11
3.7 CLASSIFICATION MODELS	12
3.8 MODELS USED IN THE PAPER	13
3.8.1. SUPPORT VECTOR CLASSIFIER	13
3.8.2 LOGISTIC REGRESSION	14
3.8.3 DECISION TREE CLASSIFIER	15
3.8.4 GRADIENT BOOSTING CLASSIFIER	16
3.8.5 KNN CLASSIFIER	17
CHAPTER 4	18
ANALYSES AND FINDINGS	18
4.1. OVERVIEW	18
4.2 DATA ANALYSES	18
CHAPTER 5	43
CONCLUSION	43
5.1 OVERVIEW	43
5.2. CONCLUSION	43
REFERENCES	44

Figure 1: Importing Libraries	21
Figure 2: Importing the data set	21
Figure 3: DataFrame first rows	22
Figure 4: Information on DataFrame	22
Figure 5: DataFrame Feature Summary	24
Figure 6: Reviewing the DataFrame for null values.	26
Figure 7: Column Labels	26
Figure 8: Redefining the DataFrame.....	27
Figure 9: Reviewing the DataFrame	27
Figure 10: Counting values of categorical variable feature Level.	27
Figure 11: Replacing object with numbers.	27
Figure 12: Reviewing the changes.....	28
Figure 13: Importing Seaborn Library	28
Figure 14: Plotting heatmap for correlation.....	28
Figure 15: Heatmap	29
Figure 16:Correlation between smoking and passive smoker	30
Figure 17:Correlation Heatmap between Level and other features	31
Figure 18: Heatmap between Level and all features.....	31
Figure 19: List of columns.....	31
Figure 20: Redefining DataFrame with important features	32
Figure 21: Reviewing redefined DataFrame.....	32
Figure 22: Counting the number of values in each class in Level column.	32
Figure 23:Defining the DataFrame X and y	32
Figure 24:Train_test_split.	33
Figure 25:Importing Logistic Regression.....	33
Figure 26:Testing the model	33
Figure 27: Accuracy Score calculation.	33
Figure 28: Confusion metrics command.....	34
Figure 29: Confusion Matrix for Logistic Regression	34
Figure 30: Classification Report for Logistic Regression	34
Figure 31: Importing K Nearest Neighbors.	35
Figure 32:Training the model.	35
Figure 33: Testing the model.....	35
Figure 34: Accuracy Score calculation.	35
Figure 35: Classification Report for Logistic Regression	36
Figure 36: Confusion metrics command.....	36
Figure 37: Confusion Matrix K-Nearest Neighbors.....	37
Figure 38: Importing Gradient Boosting Classifier	37
Figure 39: Training the model.	37
Figure 40: Testing the model.....	38
Figure 41: Accuracy Score calculation.	38
Figure 42: Classification Report for Gradient Boosting Classifier	38
Figure 43: Confusion metrics command.....	39
Figure 44: Confusion Matrix for Gradient Boosting Classifier	39
Figure 45: Importing Decision Tree Classifier.....	39
Figure 46: Training the model.	40

Figure 47: Testing the model.....	40
Figure 48: Accuracy Score calculation.	40
Figure 49: Classification report for Decision Tree	40
Figure 50: Confusion metrics command.....	41
Figure 51: Confusion Matrix for Decision Tree	41
Figure 52: Importing Support Vector Machine	42
Figure 53: Training the model.	42
Figure 54: Testing the model.	42
Figure 55: Accuracy Score calculation.	42
Figure 56: Classification Report for Support Vector Machine	43
Figure 57: Confusion metrics command.....	43
Figure 58: Confusion Matrix for Support Vector Machine.....	44
Figure 59: Cross Validation	44
Figure 60: 5-fold Cross-validation results.	45

CHAPTER-1

INTRODUCTION

1.1 OVERVIEW

Smoking is the biggest risk factor for developing lung cancer, which is still a serious health problem. For enhancing patient outcomes and directing suitable treatment, early identification and precise staging of lung cancer in smokers are essential. This study presents an advanced machine learning method for the early detection and prediction of lung cancer using a comprehensive dataset of information on patients with lung cancer, including their age, gender, exposure to air pollution, alcohol use, dust allergy, occupational risks, genetic risk, chronic lung disease, balanced diet, obesity, smoking, and passive smoking, as well as their chest pain, coughing up blood, fatigue, weight loss, shortness of breath, wheezing, and swallowing difficulties. Additionally, the study investigates the significance of various characteristics and offers information on possible biomarkers for the early detection of lung cancer. In summary, this work reveals major risk factors for lung cancer and underlines the promise of cutting-edge machine-learning approaches in the prediction of the illness. These predictive models can be applied to clinical practice to help medical professionals make defensible choices about early diagnosis and individualized treatment plans, ultimately improving patient outcomes and significantly lowering lung cancer mortality rates.

Lung cancer was the world's most common cancer-related mortality cause with 1.59 million fatalities from the disease in 2018. Smoking is the primary cause of lung cancer, although air pollution exposure is also a risk factor. According to a recent study, even among non-smokers, air pollution may raise the risk of lung cancer.

The investigation, which was reported in the journal Nature Medicine, examined information from more than 462,000 Chinese subjects who had been monitored for an average of six years. People who lived in high-pollution regions and people who lived in low-pollution areas were split into two groups of participants. Age, gender, exposure to air pollution, alcohol use, dust allergy, workplace dangers, genetic risk, chronic lung disease, balanced diet, obesity, smoking status, status as a passive smoker, fatigue levels, weight loss, shortness of breath, wheezing, swallowing difficulty, clubbing of fingernails, frequent colds, dry coughs, and snoring are all details about patients with lung cancer that are included in this dataset. This study will undertake a thorough comparison examination of several machine learning techniques, such as support vector machines, random forests, logistic regression, and gradient boosting machines, in order to determine the best-performing model. Cross-validation and hyperparameter optimization were used to fine-tune the models for maximum performance.

1.2 PROPOSED WORK

The main objective of this study is to create a machine-learning model that can accurately forecast lung cancer cases using a variety of datasets. The model should perform better in terms of lung cancer stage and early detection. The model's objective is to highlight the most significant risk factors associated with the development of lung cancer. Finding these components can boost efforts at prevention and aid in understanding the disease's underlying causes.

The main objectives of the proposed work are to:

1. Collect an appropriate dataset and preprocess the data to guarantee its quality, manage missing values, and conduct feature scaling.
2. Apply feature selection methods to the dataset to find the most relevant predictors. The model will be more effective and concentrate on the most important features if its dimensionality is decreased.
3. Utilize cross-validation and hyperparameter optimisation to optimise the performance of the chosen model.
4. To confirm the model's dependability and generalizability, evaluate its performance on an independent dataset.

1.3 RESEARCH QUESTIONS

- How does the interpretability study of the model's forecasts further our knowledge of the causes and course of lung cancer?
- What is the model's accuracy in early detection and its ability to forecast various lung cancer stages?
- What are the main variables that increase the chance of developing lung cancer and how do they affect the model's predictions?
- Does the performance of the model change depending on demographic traits like age, gender, and smoking history?

1.4 PROJECT REQUIREMENTS

- A relevant and accurate dataset for the analyses.
- A plan of action to complete the proposed work within the time.
- Technical Knowledge for the coding part.
- Detailed knowledge of machine learning algorithms.

- The related articles, journals, and other publications to gather the literature review about the work.

1.5 PROJECT RISKS

- Sometimes the coding part can take up more time(development of errors) and it may exceed the time allocated in the schedule for the coding part.
- The machine learning model runs the danger of overfitting during optimisation, which occurs when the model performs very well on the training data but fails to generalise to fresh, untried data. Poor performance in real-world applications could be the outcome.

CHAPTER-2 LITERATURE REVIEW

2.1 BACKGROUND STUDY

Early detection of lung cancer greatly enhances the odds of effective therapy. Machine learning models are able to spot small irregularities that human observers would overlook. Early identification and treatment of lung cancer might be revolutionised by the use of machine learning techniques. Realising the full potential of these algorithms will require resolving issues with data, interpretability, and practical deployment as the area develops. Machine Learning models like Classification and Ensemble models are employed to predict Lung cancer. It is now possible to determine if someone has lung cancer or not using input data from sources like patient information or medical pictures.

2.2 CLASSIFICATION MODELS

Machine learning models known as classification models are created to predict the categorical class labels of input data based on the correlations and patterns discovered in training data. In other words, new instances are classified into specified classes or categories using these models. Machine learning's foundational job is classification, and classification models are essential for addressing a variety of real-world issues, including sentiment analysis, spam email detection, picture identification,

and more. In other words, classification algorithms categorise new, unobserved occurrences into specified groups by learning patterns from labelled data. The classifications might be "positive" (lung cancer present) and "negative" (no lung cancer) in the context of lung cancer prediction. Convolutional Neural Networks (CNNs) for image-based predictions, Support Vector Machines (SVMs) for clinical and feature-based predictions, and ensemble methods like Random Forests and Gradient Boosting algorithms for combining multiple data sources are a few examples of classification models used for the prediction of lung cancer.

According to Radnath Patra's work, the RBF classifier is the best classification method for predicting lung cancer. On data pertaining to lung cancer, the RBF classifier's accuracy was determined to be 81.25%. Therefore, it can be expected from the research that accuracy will be further enhanced with a proper feature selection technique, an integrated approach with other supervised learning processes, and a modified functional approach in RBF. In the study, preexisting data is transformed into a format that is appropriate for categorization using several classifier approaches. Ten cross-validation methods are used in the classifier methodology. Both the Jupiter platform in Python and the Weka tool, version 3.6, were used in the paper's data analysis procedure. Weka is an open-source program used for data visualisation, regression, clustering, and classification. Perceptron, Naive Bayes, Decision Tree, Logistic Regression, K Nearest Neighbour, Artificial Network, and Support Vector Machine are a few examples of different learning classifiers. The RBF classifier is often chosen among other classifiers, the author concluded. This is because it has the best classification accuracy, with 26 of its 32 occurrences properly categorised and only 6 wrongly classified (Patra, 2020).

To forecast the Ensembles for Early Stage Prediction of Lung Cancer, Muhammad Imran Faisal et al. offered a number of classifiers, including Naive Bayes, Multi-Layer Perceptrons, Decision Trees, Support Vector Machines, and Decision Trees. Additionally, the performance is contrasted with those of well-known ensembles like Random Forest. Gradient-boosted Tree beat all other individual and ensemble classifiers, according to performance tests of machine learning algorithms, and obtained 90% accuracy (Faisal *et al.*, 2018). Gradient Boosting Machines (GBM), also known as Gradient Boosting Trees (GBT), is a strong ensemble learning technique that is useful for predicting lung cancer. Due to its capacity to manage complicated connections in data, handle both numerical and categorical characteristics, and offer high predictive accuracy, it is especially well-suited for applications like lung cancer prediction. The family of ensemble techniques includes gradient-boosting trees, which combine several weak learners to produce a powerful prediction model. Complex non-linear interactions between features can be represented using gradient-boosting trees. Unbalanced datasets, which are typical in medical applications, can be handled with gradient-boosting trees. Since there are frequently fewer positive instances in lung cancer datasets than negative ones, GBM can modify the learning process to take this into account.

In their article, Faezeh Hosseinzadeh et al. offered a set of machine learning models made up of seven SVM models, three ANN models, and two NB models that were applied to the original database and new models that were produced using attribute

weighting methods. Using feature extraction, feature selection, and prediction models, the author describes a diagnostic approach based on the structural and physicochemical properties of proteins implicated in both types of tumors that are deduced from sequencing data. In the paper, Faezeh Hosseinzadeh et al. offered machine learning models made up of seven SVM models, three ANN models, and two NB models that were applied to the original database and new models produced from attribute weighting models. Using feature extraction, feature selection, and prediction models, the author describes a diagnostic approach based on the structural and physicochemical properties of proteins implicated in both types of tumors that are deduced from sequencing data. To assess the model's accuracy, they utilised 10-fold cross and wrapper validation(Faezeh Hosseinzadeh et al, 2013).

In machine learning, "wrapper validation" refers to the practice of assessing the performance of one model using a different, external model. It's a method for determining how effectively a machine learning model will function in use, particularly in real-world situations. This sort of validation requires a more thorough review procedure and goes beyond the typical cross-validation or holdout validation techniques. This sort of validation requires a more thorough review procedure and goes beyond the typical cross-validation or holdout validation techniques.

In his journal machine learning model, Anil Kumar et al employed support vector machines (SVMs). Lung cancer patients are categorised according to their symptoms using an SVM classifier, and the Python programming language is also used to advance the model's implementation. On Comparing the models Anil Kumar et al suggested model to the SVM and SMOTE techniques already in use. Comparing the new approach to the current one yields a 98.8% accuracy rate. The best result on the dataset was achieved using SVM with SMOTE resampling on two iterations of the Lung Cancer dataset. This technique outperforms prior ones in that it yields the highest value for each of the evaluated parameters. The best result on the dataset was achieved using SVM with SMOTE resampling on two iterations of the Lung Cancer dataset. This technique outperforms prior ones in that it yields the highest value for each of the evaluated parameters. The author claims that running SVM and SMOTE twice on the same dataset is the best method to use them(Anil Kumar et al,2022).

CHAPTER-3

METHODOLOGY

3.1 OVERVIEW

The term "Smoker Lung Cancer Stage Detection Model" most likely refers to a model that uses machine learning or artificial intelligence to forecast the stage of lung cancer in smokers. An overview of the methodology to complete the analysis task is given below.

1. Goal-setting and problem definition: Clearly state the issue to use data analysis to address. Determine the aims and objectives of your analysis. Also, mention what are you hoping to discover or accomplish with the data.
2. Data GatheringCollect important data from numerous sources. Both structured and unstructured data, such as text Structured data may come from databases, spreadsheets, or APIs.In this paper structured data is used.
3. Data preparation and cleaning: Deal with missing values, outliers, and inconsistencies to clean up the dataset.

Prepare the data for analysis by transforming and preprocessing it, such as by normalising numerical features, transforming categorical variables into numerical representations, and dividing the data into training and test sets.

4. EDA: Exploratory Data Analysis To learn more about the properties of the dataset, explore it visually and quantitatively. To comprehend the distribution of the data, the relationships between the variables, and probable patterns, create summary statistics, histograms, scatter plots, and other visualisations.

5. Feature engineering: Build new features from current ones that might offer extra information or boost prediction accuracy. Pick pertinent characteristics that will probably have a big influence on the analysis or forecast.

6. Data modelling and analysis: Utilise the correct statistical or machine learning methods in accordance with the objectives of your investigation. Depending on the task and the type of data, this might involve regression, classification, clustering, time series analysis, or natural language processing. Models are validated using testing data after being trained using training data.

7. Results interpretation: Consider how the analysis' findings relate to your problem statement. Observe, too What new information has the data provided? Are there any suggested solutions?

8. Conclusion and Actionable Insights: Summarise the analysis's key results, conclusions, and practical advice. Address any analysis's shortcomings and make recommendations for more study or advancements.

9. Documentation: Record every step of the process, including the data sources, cleaning strategies, analytic methods, and code employed. This documentation is useful for sharing and repeatability.

3.2 DEFINE THE PROBLEM

The goal of the "Smoker Lung Cancer Stage Detection Model" is to create a prediction model that can precisely identify the stage of lung cancer in people who smoke based on pertinent medical data, which is likely to include medical pictures and perhaps additional patient information (Here patient information is used). Lung cancer's phases mainly describe how far the disease has gone within the lungs and perhaps to other organs of the body.

The main objective is to create an artificial intelligence or machine learning model that can examine medical data and make precise predictions about the stage of lung cancer in smokers. This may have significant effects on prognosis and treatment planning.

The following are the main elements of the identified problem:

A prediction algorithm should be created to categorise the stage of lung cancer in smokers based on pertinent patient information.

Benefits of this analyses:

To aid medical professionals in making wise choices regarding treatment plans.

It helps with early diagnosis and intervention, which would result in better patient outcomes.

Data preparation, feature extraction from patient data, the use of suitable machine-learning techniques, and meticulous validation and testing methods would all be required to construct a stable and accurate machine-learning model to address this issue. To guarantee patient safety and the model's usefulness in a clinical environment, it is crucial to work with medical specialists and abide by ethical standards throughout the development process.

3.3 DATA COLLECTION

Kaggle is a well-known site for datasets, but before utilising the data for any study, it is essential to ensure its validity and quality.

<https://www.kaggle.com/code/kxndxkxvw/cause-of-lung-cancer-visualization/input>

3.4 DATA PRE-PROCESSING

Data preprocessing, which entails cleaning, converting, and organising the data to make it appropriate for analysis, is a crucial stage in every data analysis effort. A data analysis project involves cleaning the data to deal with missing numbers, outliers, and discrepancies. Techniques including imputation (filling in missing information), deleting duplicate entries, and addressing outliers could be used in this stage. Making the data more analytically acceptable is a vital step in any project involving data analysis. Scaling features, storing categorical variables, and producing newly derived variables (such as calculating ratios or percentages) are examples of common transformations. Choosing whether to maintain, amend, or eliminate outliers depending on their potential influence on your analysis is also done. Outliers have the potential to distort results and impact conclusions. Effective data prior to treatment lays the foundation for insightful analysis and insights. The particular preparation stages you take will be determined by the nature of your data, your analytic goals, and the tools you're using. Always validate your preprocessing options based on their influence on the analysis outcomes.

3.5 DATA DESCRIPTION

- ❖ Patient Id: This feature is likely an identifier assigned to each patient in the dataset. It's a unique identifier that distinguishes different individuals.
- ❖ Age: Represents the age of the patient. Age can be an important factor in lung cancer detection as the risk of developing lung cancer increases with age.
- ❖ Gender: Indicates the gender of the patient (e.g., male or female). Gender can influence various aspects of health, including lung cancer risk and prevalence.
- ❖ Air Pollution: Reflects the level of air pollution that the patient is exposed to. Air pollution is a known environmental factor that can contribute to the development of lung cancer.
- ❖ Alcohol Use: Indicates whether the patient uses alcohol. Alcohol consumption can have varying effects on health, including potential links to lung cancer.
- ❖ Dust Allergy: Indicates whether the patient has an allergy to dust. Allergies and respiratory conditions might be associated with lung cancer or contribute to symptom severity.
- ❖ Occupational Hazards: Reflects whether the patient is exposed to occupational hazards that could potentially increase the risk of lung cancer. Certain occupations involve exposure to carcinogens.

- ❖ Genetic Risk: Reflects genetic factors that could contribute to the patient's risk of developing lung cancer. Genetic predisposition plays a role in cancer susceptibility.
- ❖ Chronic Lung Disease: Indicates whether the patient has a chronic lung disease, such as chronic obstructive pulmonary disease (COPD). Chronic lung diseases are associated with an increased risk of lung cancer.
- ❖ Balanced Diet: Indicates whether the patient follows a balanced diet. Nutritional factors can play a role in overall health, which in turn might affect the risk of developing diseases like cancer.
- ❖ Obesity: Reflects whether the patient is obese. Obesity is associated with various health issues, and it might influence the risk of certain cancers.
- ❖ Smoking: Indicates whether the patient is a smoker. Smoking is the most significant risk factor for lung cancer.
- ❖ Passive Smoker: Indicates whether the patient is exposed to secondhand smoke. Passive smoking can contribute to lung cancer risk.
- ❖ Chest Pain, Coughing of Blood, Fatigue, Weight Loss, Shortness of Breath, Wheezing, Swallowing Difficulty, Clubbing of fingernails, Frequent Cold, Dry Cough, and Snoring: These symptoms and indicators are relevant in the context of lung cancer. They can help in identifying potential signs of the disease.
- ❖ Level: This could be the target variable indicating the severity or likelihood of lung cancer based on the given features. It might have different categories (e.g., low risk, moderate risk, high risk).

3.6 DATA MODELLING

In a data analysis project, data modelling entails developing and training machine learning models to acquire insights, make predictions, or solve specific issues using your dataset. Several essential phases are involved in transforming raw data into actionable insights. An outline of the data modelling process in a data analysis project is provided below:

Data Preparation: The raw data should be cleaned and preprocessed. Handle missing numbers, outliers, and irregularities in format. Using approaches such as one-hot encoding or label encoding, convert category data into numerical representations. For supervised learning tasks, divide the data into features (X) and target variables (y).

Feature Engineering: To extract important information from the data, create new features or change existing ones. Combining, scaling, or extracting significant features from existing variables may be involved. Feature development can significantly impact model performance.

Data division: Divide the dataset into subgroups for training, validation, and testing. The training set is used to train the model, the validation set is used to fine-tune the hyperparameters, and the testing set is used to evaluate the model's ultimate performance.

Model Choice: Select the best machine learning algorithm for your unique assignment. The algorithm you choose is determined by the type of data (classification, regression, clustering, and so on), the quantity of the dataset, and the intended conclusion.

Model Validation: On the training data, train the chosen model. During this phase, the model uses optimisation techniques and loss functions to understand the connections between features and the target variable.

Tuning Hyperparameters: To improve the model's performance, fine-tune its hyperparameters. Hyperparameters are factors that govern the learning process and algorithm behaviour, such as learning rate, regularisation strength, and decision tree depth.

Model Assessment: Utilise relevant assessment measures to assess the model's performance. Accuracy, precision, recall, and F1-score. To choose the most appropriate metrics, consider the business or research environment.

Interpretation of the Model: Understand the link between input characteristics and predictions by interpreting the model's output. Can examine feature significance, coefficients, decision routes, and other explanations depending on the model.

3.7 CLASSIFICATION MODELS

Classification models are frequently employed for lung cancer detection because lung cancer detection is primarily a binary classification problem, given certain qualities or attributes of a patient, we wish to categorise them into one of two groups: those who have lung cancer and those who do not. Here are some of the reasons why categorization models are a good fit for this task:

The Problem Is Binary: Lung cancer detection entails determining whether a person has lung cancer (positive class) or not (negative class). Classification models are built to perform binary or multi-class classification tasks.

Predictive modelling: Modelling that makes predictions based on fresh, unforeseen data is known as classification. Classification models may learn patterns from past data. A machine may learn to distinguish between patterns linked to lung cancer and patterns linked to a healthy condition by being trained on a labelled dataset of individuals with known lung cancer status.

Interpretable Insights: Classification models can shed light on the elements that go into a patient's lung cancer diagnosis. To determine which characteristics are most predictive of the risk of developing lung cancer, feature importance scores or coefficients can be used.

Risk assessment frequently makes use of lung cancer detection. The model's predictions can be used by medical experts to gauge a patient's chance of getting lung cancer. This knowledge can direct future diagnostic procedures and treatment schemes.

Classification models may be used in screening programmes to help find people who might have a greater risk of developing lung cancer. Early detection is essential for successful treatment and better results.

Scalability and effectiveness: Classification methods are effective at processing enormous amounts of patient data. They are appropriate for large-scale screenings in

clinical settings because they can swiftly analyse the characteristics of a patient and produce a categorization result.

Clinical Decision Support: Classification models can assist clinical decision-making by offering a numerical risk score or a binary prediction. When diagnosing patients, doctors might utilise these forecasts as supplementary data.

Classification models may be updated as new data becomes available, which leads to ongoing improvement. The model's performance may be updated and improved as new patient data is gathered and labelled.

3.8 MODELS USED IN THE PAPER

The models used in this paper are the Support vector Classifier, Decision Tree Classifier, GradientBoosting Classifier, Logistic Regression, and K-Neighbour Classifier.

Each of these classifiers has advantages and disadvantages, and depending on the dataset and situation at hand, they may perform better or worse. The size of the dataset, the intricacy of the connections, interpretability, and the necessity for robustness against overfitting all play a role in the classifier decision. It's crucial to carefully assess each classifier's performance while developing a lung cancer detection model using the relevant assessment metrics, such as accuracy, precision, recall, and F1-score. You may choose the classifier that is best for your particular job and dataset using this evaluation. It's also a good idea to experiment with hyperparameter adjustment to enhance the performance of each classifier.

3.8.1. SUPPORT VECTOR CLASSIFIER

A machine learning approach known as the Support Vector Classifier (SVC) is utilised largely for binary classification issues. It belongs to the Support Vector Machines (SVM) algorithm family. Finding the optimum hyperplane to divide data points into distinct classes is the goal of SVC. Finding the best hyperplane to maximise the gap between the two classes while reducing classification error is the goal of SVC. The margin is the separation between each class's closest data points and the hyperplane. Support vectors are used to refer to the data points that are closest to the hyperplane because they "support" the hyperplane's definition.

SVC searches for a hyperplane that can properly separate the two classes in the case of linearly separable data. The distance between the classes should be the widest on this hyperplane. The equation of the hyperplane establishes the decision boundary. SVC employs a method known as the "kernel trick" for data that cannot be separated linearly. In a higher-dimensional space created by kernels, the data may then be linearly separable. Radial basis function (RBF) kernels, polynomial kernels, and linear kernels are common types of kernel functions.

With SVC, a regularisation parameter (C) is added that regulates the trade-off between properly categorising training instances and gaining a greater margin. While a greater C value seeks to categorise all cases properly but may result in a narrower margin, a smaller C value permits a wider margin but may produce examples that are incorrectly

classified. When the data cannot be separated properly, SVC can manage the situation by adding a "soft margin." The algorithm can establish a compromise between increasing the margin and minimising misclassifications thanks to the soft margin, which permits some misclassification.

The SVC classifier's decision function determines how far a data point is from the hyperplane. The data point is assigned to one class if the distance is positive and the other class if the distance is negative. The trained hyperplane is used by SVC to forecast the class of fresh data points based on which side of the hyperplane they fall on.

Strengths

- In high-dimensional spaces, it is effective.
- works effectively for data that can be separated in both linear and nonlinear ways.
- can efficiently handle big datasets.
- able to change data using a variety of kernel functions.

Weaknesses:

- It is necessary to tune the hyperparameters, particularly for the kernel and regularisation parameter selection.
- might need a lot of computer power, especially for huge datasets.
- It can be difficult to interpret, especially with non-linear kernels.

In conclusion, SVC is a potent classifier that works well for a variety of classification issues. Because it can handle both linear and non-linear separation, it is adaptable for many kinds of datasets. To get the best results, however, precise hyperparameter tuning and comprehension of the ramifications of kernel selections are crucial.

3.8.2 LOGISTIC REGRESSION

For binary classification issues, a popular machine-learning approach is logistic regression. Despite its name, it is utilised for classification rather than regression. The likelihood that a given input belongs to a certain class is estimated through logistic regression. The logistic function, often known as the sigmoid function, is used in logistic regression to represent the likelihood that a data point belongs to a certain class. The logistic function's output is limited between 0 and 1, making it appropriate for describing probability. The probability p that a data point belongs to the positive class (class 1) is predicted by the logistic regression model.

$p = 1 / (1 + e^{-z})$, where z is a linear combination of the input characteristics and their associated weights, is the definition of the hypothesis function.

The expected probability is subjected to a threshold (often 0.5) before a categorization judgement is made. The data point is categorised into the positive class if p is larger than or equal to the threshold, otherwise, it is categorised into the negative class. Utilising optimisation approaches, the weights (coefficients) connected to the features are learned from the training data. It is the objective to identify the weights

that reduce a cost function, such as the logistic loss or cross-entropy loss. Logistic regression can contain regularisation terms like L1 (Lasso) or L2 (Ridge) regularisation to avoid overfitting. These terms limit large coefficients and aid in model simplification. The interpretability of logistic regression is one of its advantages. The model's coefficients show how much altering a certain attribute changes the predicted probability's log odds. Positive coefficients raise the likelihood that a class is positive, whereas negative coefficients lower the likelihood.

Strengths

The model's simplicity and interpretability are strengths.

It can shed light on the value of a feature.

It is ideal for data that can be separated linearly. It works best when there is a roughly linear

relationship between the characteristics and the output.

Weaknesses:

It assumes that features and log-odds have a linear relationship, which may not be true for complicated data.

Without feature engineering or modification, may not function effectively on significantly non-linear data.

It is task-specific to binary classification alone.

3.8.3 DECISION TREE CLASSIFIER

A well-liked machine learning approach for classification tasks is the Decision Tree Classifier. This model, which resembles a tree, decides by dividing the data into subsets according to the values of the input characteristics. A series of choices that result in a classifying conclusion are represented by a decision tree. Each leaf node in the tree represents a class label, whereas each internal node in the tree corresponds to a judgment based on a particular characteristic. The objective is to build a tree that divides the data into subsets with as few inconsistencies in the class labels as feasible. The decision Tree method selects the ideal feature and threshold at each internal node to divide the data into subsets. Based on a criterion intended to improve the subsets' purity, the decision to divide is made. Entropy and Gini impurity are typical criteria.

The likelihood of incorrectly categorising a randomly selected data point inside a subset is gauged by the Gini impurity. Greater purity (homogeneity) of the subgroup in terms of class labels is indicated by a lower Gini impurity. For each class i in the subset, it is computed as $1 - \sum(p_i^2)$.

The impurity or randomness of a subset is measured by entropy. For each class i in the subset, it is computed as $-\sum(p_i * \log_2(p_i))$. Higher purity is associated with lower entropy. Once a splitting requirement is satisfied, the Decision Tree method

constructs the tree recursively by repeatedly dividing nodes. This might be the maximum tree depth, the bare minimum of samples in a leaf, or the point at which additional splits do not significantly increase purity.

Entropy is a metric for a subset's impurity or unpredictability. It is calculated using the formula $-\sum(p_i \cdot \log_2(p_i))$ for each class i in the subset. Lower entropy is correlated with higher purity. The Decision Tree technique builds the tree recursively by continually splitting nodes after a splitting condition is met. The maximum depth of a tree, the bare minimum of samples included in a leaf, or the point at which further splits do not meaningfully improve purity are all examples of such phenomena.

It moves through the tree, classifying each internal node as it goes, starting at the root node and ending at the leaf node. The projected class for the input data is determined by the class label connected to the reached leaf node.

Strengths:

Simple to visualise and comprehend.

may record intricate connections, including non-linear ones.

Weakness:

Appropriate for numerical and category characteristics.

Possibility of overfitting, particularly with deep trees.

sensitive to subtle data variances.

If not adequately pruned, may not generalise successfully to unknown data.

3.8.4 GRADIENT BOOSTING CLASSIFIER

An ensemble machine learning approach for classification problems is called a Gradient Boosting Classifier. It is a potent strategy that builds a strong predictive model by combining the predictions of several weak learners, often decision trees.

Gradient Boosting successively creates an ensemble of ineffective learners, with each new learner attempting to fix the mistakes committed by the preceding ones. It concentrates on enhancing the areas where the existing model fails. To prevent overfitting, the weak learner is frequently employed as a base model and is a decision tree with little depth (referred to as a "shallow tree"). These tiny trees are frequently called "stumps" or "leaf stumps."

The technique begins with a basic model that offers an initial prediction, often a shallow tree. The flaws of the earlier models are fixed by the later models. The weighted average of all weak learners' guesses makes up the final forecast. The algorithm determines the discrepancy between the actual labels and the ensemble's current predictions for each iteration. The term "residual" refers to this disparity. Then, fresh models are taught to anticipate these residuals. The amount that each new model contributes to the ensemble's prediction is managed by a learning rate. Gradient Boosting modifies the parameters of the gradient descent algorithm to minimise the

loss function (usually a loss function like the squared error for regression or deviation for classification).

The final ensemble prediction is created by combining all weak learners' predictions. The amount that each model's prediction adds to the end outcome depends on the learning rate.

To avoid overfitting, gradient boosting uses regularisation techniques. Common strategies include reducing the depth of each weak learner and training each new model via subsampling (sampling a fraction of the data).

Strengths:

A very capable and precise algorithm.

It can capture intricate interactions and linkages in the data.

It takes care of both numerical and category characteristics.

Drawbacks:

might be time-consuming and computationally demanding.

prone to overfitting if improperly calibrated.

calls for meticulous hyperparameter adjustment.

3.8.5 KNN CLASSIFIER

A machine learning technique used for classification problems is called the k-nearest Neighbours (k-NN) Classifier. Based on the majority class of a data point's k-nearest neighbours, it generates predictions. On the assumption that related data points are likely to belong to the same class, k-NN operates. A data point's classification is determined by the class labels of its closest neighbours.

To enable effective closest neighbour search during the training phase, the technique stores the feature vectors and their accompanying class labels in a data structure, frequently a tree or a spatial index. The similarity between data points is determined by k-NN using a distance metric (such as the Manhattan distance, Euclidean distance, etc.). The type of distance measure chosen relies on the nature of the data. The method determines the distances between each new data point and every other point in the training set when a new data point has to be categorised. Then, depending on the closest distances, it chooses the k-nearest neighbours.

The number of neighbours in each class among the k-nearest neighbours is counted by the k-NN algorithm. The anticipated class label for the new data point is the one that is most prevalent among the neighbours.

The hyperparameter k and its value control how many neighbours are taken into account when producing predictions. While a bigger number helps smooth the decision limits, a lesser value of k can provide noisy forecasts.

Some implementations employ a weighted voting method, wherein the forecast is more heavily influenced by neighbours who are closer to the voter. To give closer

neighbours more weight, the weights can be inversely proportional to distance. In distance weighting, neighbours are given weights based on how far away from the query location. The designation of the class that Nearby neighbours are given more weight, has a greater impact on the forecast.

Strengths:

Plain and simple to comprehend.

effective with datasets that have distinct clusters and divisions.

It can do both regression and classification tasks.

Weakness:

Sensitive to noise and outliers is a weakness.

it requires features to be properly scaled in order to support varied ranges.

computationally costly for huge datasets during prediction.

CHAPTER 4

ANALYSES AND FINDINGS

4.1. OVERVIEW

This chapter discusses more about the analysis of the data to identify patterns and trends observed in the data collected from Kaggle, the data pre-processing to ensure that the input data doesn't contain serious outliers, missing data, duplicates, redundant columns, and so on, and the process of modelling, training of models, testing, performance evaluation and validation of the results.

4.2 DATA ANALYSES

The process of data analysis or the exploratory data analysis, is conducted to identify patterns, trends, relationships between variables, and importance of various features. The Google Colaboratory platform, which we employ to write and run the Python codes and the dataset collected from Kaggle for prediction of lung cancer are the primary requirement for this data analysis and modelling part of the research. Below are the various steps taken towards the data analysis as part of the research.

```
[ ] import pandas as pd
    import numpy as np
    import matplotlib.pyplot as plt
```

Figure 1: Importing Libraries

Python has various libraries, which are typically the collection of pre-written codes or functions, which helps the programmers to extend the capabilities of the language and perform various tasks without writing the code from scratch. The various functionalities these libraries cover includes data manipulation, scientific computing, machine learning, data visualisation, web development and programming, and much more.

import pandas as pd – Imports the Pandas library, which is used for the manipulation and analysis of data and allows the researcher to perform analysis and operations multidimensional data structures such as DataFrame.

import numpy as np – Imports the NumPy library, which again is a fundamental Python library used for scientific computing, enabling the researcher to perform mathematical functions and operations on data structures such as arrays and matrices.

import matplotlib.pyplot as plt – Imports the pyplot module from the Matplotlib, a library for data visualisation using different types of charts, plots, etc.

```
[ ] lung_cancer=pd.read_csv('/content/lung_cancer.csv')
```

Figure 2: Importing the data set.

In this step, the dataset which was collected from Kaggle is being uploaded or imported to create the new DataFrame lung_cancer. The dataset file lung_cancer.csv, is a two-dimensional array with rows and columns, and the command reads the dataset and then creates the DataFrame lung_cancer.

lung_cancer.head()

	index	Patient Id	Age	Gender	Air Pollution	Alcohol use	Dust Allergy	Occupational Hazards	Genetic Risk	chronic Lung Disease	...	Fatigue	Weight Loss	Shortness of Breath	Wheezing	Swallowing Difficulty	Clubbing of Finger Nails	Frequent Cold	Dry Cough	Snoring	Level
0	0	P1	33	1	2	4	5	4	3	2	...	3	4	2	2	3	1	2	3	4	Low
1	1	P10	17	1	3	1	5	3	4	2	...	1	3	7	8	6	2	1	7	2	Medium
2	2	P100	35	1	4	5	6	5	5	4	...	8	7	9	2	1	4	6	7	2	High
3	3	P1000	37	1	7	7	7	7	6	7	...	4	2	3	1	4	5	6	7	5	High
4	4	P101	46	1	6	8	7	7	7	6	...	3	2	4	1	4	2	4	2	3	High

5 rows x 26 columns

Figure 3: DataFrame first rows

In this step, the first few rows of the DataFrame are displayed as output, and also it can be seen that the number of rows starts from 0. Here, the first five rows, whose index value ranging from 0 to 4 are returned and the 26 column headers (features or attributes) can be seen. More information on the DataFrame is collected using the code below:

```

lung_cancer.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 26 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   index                                     1000 non-null   int64
1   Patient Id                               1000 non-null   object
2   Age                                       1000 non-null   int64
3   Gender                                   1000 non-null   int64
4   Air Pollution                           1000 non-null   int64
5   Alcohol use                             1000 non-null   int64
6   Dust Allergy                            1000 non-null   int64
7   OccuPational Hazards                    1000 non-null   int64
8   Genetic Risk                            1000 non-null   int64
9   chronic Lung Disease                    1000 non-null   int64
10  Balanced Diet                           1000 non-null   int64
11  Obesity                                  1000 non-null   int64
12  Smoking                                  1000 non-null   int64
13  Passive Smoker                           1000 non-null   int64
14  Chest Pain                               1000 non-null   int64
15  Coughing of Blood                        1000 non-null   int64
16  Fatigue                                  1000 non-null   int64
17  Weight Loss                              1000 non-null   int64
18  Shortness of Breath                      1000 non-null   int64
19  Wheezing                                 1000 non-null   int64
20  Swallowing Difficulty                    1000 non-null   int64
21  Clubbing of Finger Nails                 1000 non-null   int64
22  Frequent Cold                           1000 non-null   int64
23  Dry Cough                                1000 non-null   int64
24  Snoring                                  1000 non-null   int64
25  Level                                    1000 non-null   object
dtypes: int64(24), object(2)
memory usage: 203.2+ KB

```

Figure 4: Information on DataFrame

This command is run to understand more information on the DataFrame.

RangeIndex – It is the range of the index values, that starts from 0 and here, the DataFrame contains 1000 entries, which means the range index value will be 0 – 999.

Data Columns – It gives the total number of columns in the DataFrame, which is 26.

Columns – The headers of each column (or features) in the DataFrame are displayed.

The summary of the Columns are as follows:

- **Index** – It is the index number of the entry corresponding to patients present in the dataset and is numeric data.
- **Patient Id** – The identification number of the patient and is a string.
- **Age** – It is the age of the patient, which is numeric feature.
- **Gender** - It is the gender of the patient and is a categorical feature (1 or 2 for Male or Female gender).
- **Air Pollution** – It is the level of exposure the patient has to air pollution and is again a categorical feature (1-8 levels).
- **Alcohol use** – It is the level of alcohol consumption by the patient and is a categorical feature (1-8 levels).
- **Dust Allergy** – It can be described as the dust allergy levels of the patient and is a categorical feature (1-8 levels).
- **OccuPational Hazards** - It is the level of occupational hazards that the patient might have and is again a categorical variable (1-8 levels).

- Genetic Risk – It is the genetic risk level of the patient to have the disease and is a categorical variable (1-7 levels).
- chronic Lung Disease - It is the chronic lung disease level in the patient and is a categorical feature (1-7 levels).
- Balanced Diet – It is the balanced diet levels followed by the patient and is a categorical variable (1-7 levels).
- Obesity – It is the obesity levels of the patient and is a categorical variable (1-7 levels).
- Smoking – It is the smoking habit levels of the patient and is a categorical variable (1-8 levels).
- Passive Smoker – It is passive smoking exposure the patient has and is a categorical feature (1-8 levels).
- Chest Pain – It is the chest pain levels in the patient and is a categorical feature (1-9 levels).
- Coughing of Blood – It is the blood coughing levels in the patient and is a categorical feature (1-9 levels).
- Fatigue - It is the level of fatigue experienced by the patient and is a categorical feature (1-9 levels).
- Weight Loss – It is the weight loss levels of the patient and is categorical feature (1-8 levels).
- Shortness of Breath – It is the level of shortness of breath the patient experience and is again a categorical feature (1-9 levels).
- Wheezing – It is the wheezing level experienced by the patient and is a categorical feature (1-8 levels).
- Swallowing Difficulty - It is the level of difficulty in. swallowing faced by the patient (1-8 levels).
- Clubbing of Finger Nails – It is the level of clubbing of fingers symptom observed in the patient and is a categorical feature (1-9 levels).
- Frequent Cold – It is the level of frequency in cold in the patients and also is a categorical feature (1-7 levels).
- Dry Cough – It is the level of dry cough in the patient and is a categorical variable (1-7 levels).

- Snoring – It is the level of snoring of the patient and is a categorical variable (1-7 levels).
- Level – It is the level of cancer in the patients and is a categorical variable (High, Medium, and Low)

Non-null Count – This gives the non-null count or number of non-null items in the column and helps to identify the number of missing values corresponding to the columns. All the columns had 1000 non-null count each.

Dtype – It denotes the data type of each of the column, or feature. Here, except for Patient Id and Level, which are both objects, all the other columns have 64-bit signed integers.

dtypes – It gives the summary of the data types and here there are 24 features with int64 data type while there are two features with object data type.

memory usage – It gives the value of memory used by the DataFrame, which here is 203.2+KB.

	Index	Age	Gender	Air Pollution	Alcohol use	Dust Allergy	Occupational Hazards	Genetic Risk	Chronic Lung Disease	Balanced Diet	...	Coughing of Blood	Fatigue	Weight Loss	Shortness of Breath	Wheezing	Swallowing Difficulty	Clubbing of Finger Nails	Frequent Cold	Dry Cough	Smoking
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	...	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	499.500000	37.174000	1.402000	3.8400	4.563000	5.165000	4.840000	4.580000	4.380000	4.491000	...	4.859000	3.856000	3.855000	4.240000	3.777000	3.746000	3.923000	3.536000	3.853000	2.926000
std	288.819436	12.005493	0.490547	2.0304	2.620477	1.980833	2.107805	2.126999	1.848518	2.135528	...	2.427965	2.244616	2.206546	2.285087	2.041921	2.270383	2.388048	1.832502	2.039007	1.474686
min	0.000000	14.000000	1.000000	1.0000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	...	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
25%	249.750000	27.750000	1.000000	2.0000	2.000000	4.000000	3.000000	2.000000	3.000000	2.000000	...	3.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000
50%	499.500000	36.000000	1.000000	3.0000	5.000000	6.000000	5.000000	5.000000	4.000000	4.000000	...	4.000000	3.000000	3.000000	4.000000	4.000000	4.000000	3.000000	3.000000	4.000000	3.000000
75%	749.250000	45.000000	2.000000	6.0000	7.000000	7.000000	7.000000	7.000000	6.000000	7.000000	...	7.000000	5.000000	6.000000	6.000000	5.000000	5.000000	5.000000	5.000000	6.000000	4.000000
max	999.000000	73.000000	2.000000	8.0000	8.000000	8.000000	8.000000	7.000000	7.000000	7.000000	...	9.000000	9.000000	8.000000	9.000000	8.000000	8.000000	9.000000	7.000000	7.000000	7.000000

Figure 5: DataFrame Feature Summary

This step describes the DataFrame, based on its various summary statistics values. All the features with int64 data type, are described while features having non numerical data type object which are Patient Id and Level are not described.

count – The number of non-null values in the respective feature column and here it can be observed that all the features have a count of 1000.

mean – The average or mean is measured as the sum of all values divided by total number or count of values, which is 1000. Here the mean of the age is 37.17 which shows that the average age distribution in the DataFrame is around 37 years.

std - The standard deviation is the deviation from the mean value and here standard deviation for age 12.006.

min – The minimum value in the feature distribution of the DataFrame. Here, the minimum age of the patient is 14 years.

25% - Q1 or first quartile or 25 percentile is the value of the feature below which the 25 percent of the patients lies, and here, the 25 percent of the patient lies below an age 27.75 years.

50% - Q2 or second quartile or 50 percentile is also called as the median is the value of the feature below which the 50 percent of the patients lies, and here, the 50 percent of the patient lies below an age 36 years.

75% - Q3 or third quartile or 75 percentile is the value of the feature below which the 75 percent of the patients lies, and here, the 75 percent of the patient lies below an age 45 years.

max - The maximum value in the feature distribution of the DataFrame. Here, the maximum age of the patient is 73 years.

This is similar for other features as well.

```
lung_cancer.isnull().sum()
```

index	0
Patient Id	0
Age	0
Gender	0
Air Pollution	0
Alcohol use	0
Dust Allergy	0
OccuPational Hazards	0
Genetic Risk	0
chronic Lung Disease	0
Balanced Diet	0
Obesity	0
Smoking	0
Passive Smoker	0
Chest Pain	0
Coughing of Blood	0
Fatigue	0
Weight Loss	0
Shortness of Breath	0
Wheezing	0
Swallowing Difficulty	0
Clubbing of Finger Nails	0
Frequent Cold	0
Dry Cough	0
Snoring	0
Level	0

dtype: int64

Figure 6: Reviewing the DataFrame for null values.

The number of null values if any in the DataFrame is displayed in this step and it can be observed there are no null values or missing data in the DataFrame.

```
lung_cancer.columns

Index(['index', 'Patient Id', 'Age', 'Gender', 'Air Pollution', 'Alcohol use',
      'Dust Allergy', 'OccuPational Hazards', 'Genetic Risk',
      'chronic Lung Disease', 'Balanced Diet', 'Obesity', 'Smoking',
      'Passive Smoker', 'Chest Pain', 'Coughing of Blood', 'Fatigue',
      'Weight Loss', 'Shortness of Breath', 'Wheezing',
      'Swallowing Difficulty', 'Clubbing of Finger Nails', 'Frequent Cold',
      'Dry Cough', 'Snoring', 'Level'],
      dtype='object')
```

Figure 7: Column Labels

In this step the column labels or feature names are displayed. This is done to see what columns are relevant and what is to be dropped.

```
[ ] lung_cancer=lung_cancer[['Age', 'Gender', 'Air Pollution', 'Alcohol use',
      'Dust Allergy', 'OccuPational Hazards', 'Genetic Risk',
      'chronic Lung Disease', 'Balanced Diet', 'Obesity', 'Smoking',
      'Passive Smoker', 'Chest Pain', 'Coughing of Blood', 'Fatigue',
      'Weight Loss', 'Shortness of Breath', 'Wheezing',
      'Swallowing Difficulty', 'Clubbing of Finger Nails', 'Frequent Cold',
      'Dry Cough', 'Snoring', 'Level']]
```

Figure 8: Redefining the DataFrame

The DataFrame is redefined without the columns such as index and Patient Id in this step. The index number or patient identification number are irrelevant to the prediction of lung cancer.

lung_cancer.head()

	Age	Gender	Air Pollution	Alcohol use	Dust Allergy	OccuPational Hazards	Genetic Risk	chronic Lung Disease	Balanced Diet	Obesity	...	Fatigue	Weight Loss	Shortness of Breath	Wheezing	Swallowing Difficulty	Clubbing of Finger Nails	Frequent Cold	Dry Cough	Snoring	Level
0	33	1	2	4	5	4	3	2	2	4	...	3	4	2	2	3	1	2	3	4	Low
1	17	1	3	1	5	3	4	2	2	2	...	1	3	7	8	6	2	1	7	2	Medium
2	35	1	4	5	6	5	5	4	6	7	...	8	7	9	2	1	4	6	7	2	High
3	37	1	7	7	7	7	6	7	7	7	...	4	2	3	1	4	5	6	7	5	High
4	46	1	6	8	7	7	7	6	7	7	...	3	2	4	1	4	2	4	2	3	High

5 rows x 24 columns

Figure 9: Reviewing the DataFrame

The first five rows or entries of the redefined DataFrame is displayed in this step and we can see that the index and Patient Id columns were dropped.

```
lung_cancer['Level'].value_counts()

High      365
Medium    332
Low       303
Name: Level, dtype: int64
```

Figure 10: Counting values of categorical variable feature Level.

The categorical variable Level is classified into three categories, High, Medium and Low and the number or count of values in each of these classes is calculated using

the value_counts function and the result is displayed. 365 entries are of High class, 332 entries of Medium class and 303 entries of Low class were found.

```
[ ] lung_cancer=lung_cancer.replace({'Level':{'Low': 1, 'Medium': 2, 'High': 3}})
```

Figure 11: Replacing object with numbers.

In this step the class names of the feature, Level, which are High, Medium and Low are replaced using 3, 2 and 1 respectively. This is done to change the data type of the feature from object to numeric.

lung_cancer.head()

	Age	Gender	Air Pollution	Alcohol use	Dust Allergy	Occupational Hazards	Genetic Risk	chronic Lung Disease	Balanced Diet	Obesity	...	Fatigue	Weight Loss	Shortness of Breath	Wheezing	Swallowing Difficulty	Clubbing of Finger Nails	Frequent Cold	Dry Cough	Scoring Level	
0	33	1	2	4	5	4	3	2	2	4	—	3	4	2	2	3	1	2	3	4	1
1	17	1	3	1	5	3	4	2	2	2	—	1	3	7	8	6	2	1	7	2	2
2	35	1	4	5	6	5	5	4	6	7	—	8	7	9	2	1	4	6	7	2	3
3	37	1	7	7	7	7	6	7	7	7	—	4	2	3	1	4	5	6	7	5	3
4	46	1	6	8	7	7	7	6	7	7	—	3	2	4	1	4	2	4	2	3	3

5 rows x 24 columns

Figure 12: Reviewing the changes.

The DataFrame is reviewed in this step to see if the replacement made in the Level feature is reflecting in the DataFrame. The number of columns and rows remain unchanged.

```
[ ] import seaborn as sns
    sns.set()
```

Figure 13: Importing Seaborn Library

The Seaborn is a data visualisation library that allows to plot complex data visualisations which can be presented as simple static plots, interactive and informative plots or even animated and visually appealing plots.

```
plt.figure(figsize = (18,10))
sns.heatmap(lung_cancer.corr(), annot=True)
plt.show()
```

Figure 14: Plotting heatmap for correlation

A heatmap is a data visualisation plot, which is used to depict the relationship between different features within the DataFrame, but especially here it is done to understand the relationship or correlation between Level and other features. The relationship can be a positive correlation, a negative correlation or no correlation. Positive correlation means that the increase in one feature is associated with the increase in the other feature. Similarly, negative correlation means that increase in one of the features associated with the decrease in the other feature and vice versa. No correlation or zero correlation means that the increase or decrease in a feature does not affect the other feature. The range of correlation is from -1 to 1, where -1 indicates perfect negative correlation and 1 indicates perfect positive correlation and 0 means no

correlation. The values nearing to -1 or 1 means high correlation and values closer to 0 means poor correlation.

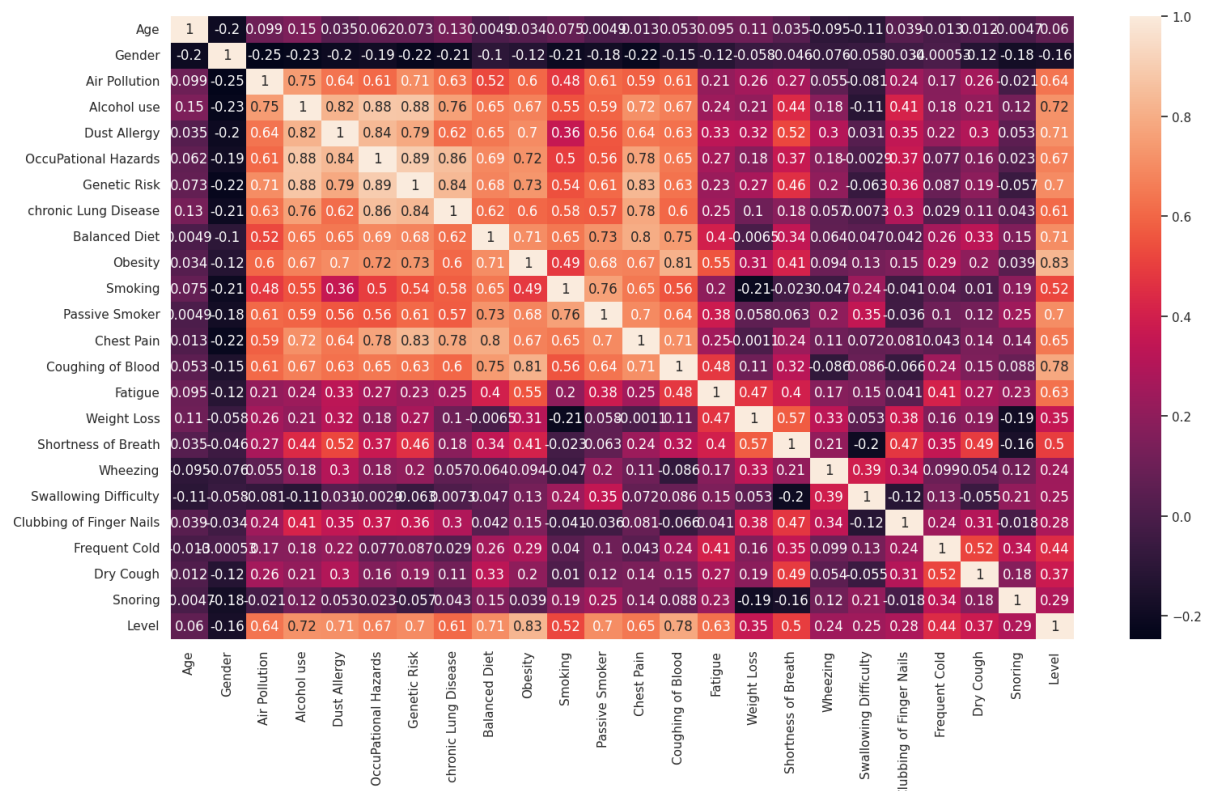


Figure 15: Heatmap

Heatmap here contains the relationship between each of the features and is easy to understand. The value of relationship ranges from -0.2 (as it is the maximum value of negative correlation) to 1, which is the maximum value of positive correlation. But since the aim is to predict lung cancer, the feature Level, which is the target variable, and its relationship other features are interpreted.

Age – Low positive correlation (0.06), which means that age doesn't determine the change in the Level value.

Gender – Low negative correlation (-0.16), which means that gender doesn't have much impact on the Level value.

Air Pollution - High positive correlation (0.64), which means that air pollution levels do have a strong impact on the Level value.

Alcohol use - High positive correlation (0.72), which means that alcohol use levels does have a strong impact on the Level value.

Dust Allergy - High positive correlation (0.71), which means that dust allergy levels do have a strong impact on the Level value.

Occupational Hazards - High positive correlation (0.67), which means that occupational hazards levels do have a strong impact on the Level value.

Genetic Risk - High positive correlation (0.7), which means that genetic risk levels do have a strong impact on the Level value.

chronic Lung Disease - High positive correlation (0.61), which means that chronic lung disease levels do have strong impact on the Level value.

Balanced Diet - High positive correlation (0.71), which means that balanced diet levels do have strong impact on the Level value.

Obesity - High positive correlation (0.83), which means that obesity levels do have strong impact on the Level value.

Smoking - High positive correlation (0.52), which means that smoking levels do have strong impact on the Level value.

Passive Smoker - High positive correlation (0.7), which means that passive smoking levels do have an impact on the Level value.

Chest Pain - High positive correlation (0.65), which means that chest pain levels do have strong impact on the Level value.

Coughing of Blood - High positive correlation (0.78), which means that coughing of blood levels do have strong impact on the Level value.

Fatigue - High positive correlation (0.63), which means that fatigue levels do have strong impact on the Level value.

Weight Loss - Positive correlation (0.35), which means that weight loss levels do have an impact on the Level value, but not too strong.

Shortness of Breath - Strong positive correlation (0.5), which means that shortness of breath levels does have strong impact on the Level value.

Wheezing - Positive correlation (0.24), which means that wheezing levels do have an impact on the Level value, though not too strong.

Swallowing Difficulty - Positive correlation (0.25), which means that swallowing difficulty levels do have an impact on the Level value, though not too strong.

Clubbing of Finger Nails - Positive correlation (0.28), which means that clubbing of finger nails levels does have an impact on the Level value, though not too strong.

Frequent Cold - Positive correlation (0.44), which means that frequent cold levels do have an impact on the Level value, though not too strong.

Dry Cough - Positive correlation (0.37), which means that dry cough levels do have an impact on the Level value, though not too strong.

Snoring - Positive correlation (0.29), which means that snoring levels do have an impact on the Level value, though not too strong.

```
[ ] lung_cancer['Smoking'].corr(lung_cancer['Passive Smoker'])  
  
0.7616215035339878
```

Figure 16:Correlation between smoking and passive smoker

The correlation between the feature Smoking and Passive Smoker was calculated that being a smoker also increases the risk of being a passive smoker.

```
sns.heatmap(lung_cancer.corr()[['Level']].sort_values(by='Level', ascending=False), vmin=-1, vmax=1, annot=True, cmap='GnBu')
```

Figure 17:Correlation Heatmap between Level and other features

This command will display the correlation between the various features and Level feature in the descending order and helps in understanding the feature importance.

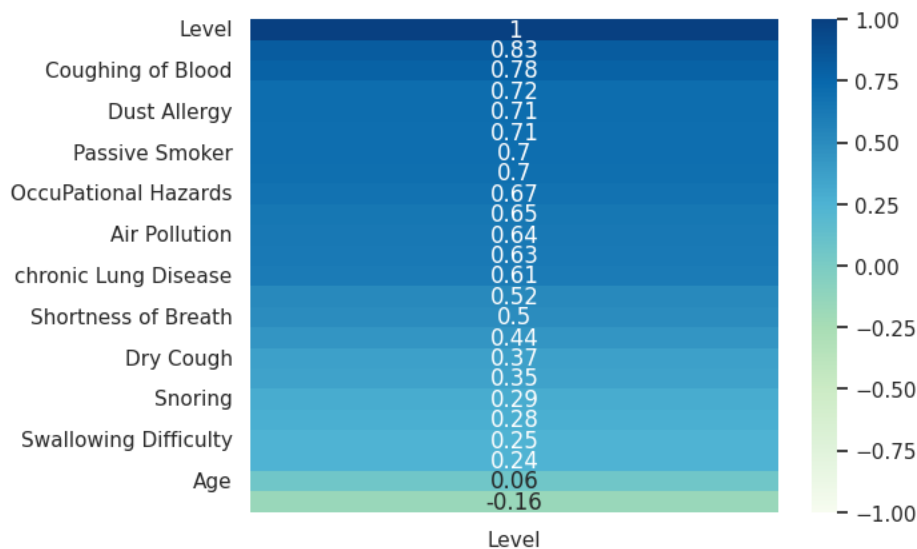


Figure 18: Heatmap between Level and all features

The heatmap between level and all the column labels or features is displayed in the order of decreasing correlation will help in identifying the features that are important and is therefore, critical in predicting the risk of lung cancer. Coughing of blood has the highest value of correlation, followed by dust allergy, passive smoker, occupational hazards, air pollution, chronic lung disease, and so on.

```
[ ] lung_cancer.columns
```

```
Index(['Age', 'Gender', 'Air Pollution', 'Alcohol use', 'Dust Allergy',  
      'OccuPational Hazards', 'Genetic Risk', 'chronic Lung Disease',  
      'Balanced Diet', 'Obesity', 'Smoking', 'Passive Smoker', 'Chest Pain',  
      'Coughing of Blood', 'Fatigue', 'Weight Loss', 'Shortness of Breath',  
      'Wheezing', 'Swallowing Difficulty', 'Clubbing of Finger Nails',  
      'Frequent Cold', 'Dry Cough', 'Snoring', 'Level'],  
      dtype='object')
```

Figure 19: List of columns

The index list of columns is created once again for the DataFrame, to identify and define the new list consisting important features that have strong correlation to the levels of lung cancer.

```
[ ] lung_cancer=lung_cancer[['Age','Coughing of Blood','Dust Allergy','Passive Smoker','Occupational Hazards','Air Pollution','chronic Lung Disease','Shortness of Breath','Dry Cough','Snoring','Swallowing Difficulty','Level']]
```

Figure 20: Redefining DataFrame with important features

The key important features identified from the correlation heatmap is used to define the DataFrame lung_cancer and the revised DataFrame will consists of columns Age, Coughing of Blood, Dust Allergy, Passive Smoker, Occupational Hazards, Air Pollution, chronic Lung Disease, Shortness of Breath, Dry Cough, Snoring, Swallowing Difficulty, and Level.

lung_cancer.head()

	Age	Coughing of Blood	Dust Allergy	Passive Smoker	Occupational Hazards	Air Pollution	chronic Lung Disease	Shortness of Breath	Dry Cough	Snoring	Swallowing Difficulty	Level		
0	33		4	5	2		4	2	2	3	4	3	1	
1	17		3	5	4		3	3	2	7	7	2	6	2
2	35		8	6	3		5	4	4	9	7	2	1	3
3	37		8	7	7		7	7	7	3	7	5	4	3
4	46		9	7	7		7	6	6	4	2	3	4	3

Figure 21: Reviewing redefined DataFrame.

This step is done to review the redefined DataFrame lung_cancer have all the required column labels or attributes. The top 5 rows are displayed and can be observed that there are 12 columns in this DataFrame.

```
[ ] lung_cancer['Level'].value_counts()

3      365
2      332
1      303
Name: Level, dtype: int64
```

Figure 22: Counting the number of values in each class in Level column.

This step counts the count of each value 3, 2 and 1 in the column Level using the value_counts function and it can be observed that there are 365, 332 and 303 values for the categories 3, 2 and 1 respectively.

```
[ ] X=lung_cancer.drop('Level',axis=1)

y=lung_cancer['Level']
```

Figure 23:Defining the DataFrame X and y

In this step, the X and y DataFrames are defined from the lung_cancer DataFrame. First, the X DataFrame is defined by dropping the feature Level and then the y

DataFrame is defined with only feature Level as a method of extracting the target feature or variable.

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.30,random_state=1)
```

Figure 24:Train_test_split.

The train_test_split function is imported from the sklearn.model_selection module within the sklearn library, which is a very commonly used library for ML modelling. Here, the X and y DataFrames are split using this function into train subset and test subset, where the size of the test size is selected as 0.30 or 30 percent of the DataFrame.

```
[ ] from sklearn.linear_model import LogisticRegression
    model_1 = LogisticRegression(solver='liblinear')
    model_1.fit(X_train,y_train)
```

```
▼ LogisticRegression
LogisticRegression(solver='liblinear')
```

Figure 25:Importing Logistic Regression

In this step, the Logistic Regression model, which is a classic or linear model, which is present in the sklearn's linear model module within the sklearn library is imported. The solver parameter is set to liblinear and the model is trained.

```
[ ] y_prediction_1 = model_1.predict(X_test)
```

Figure 26:Testing the model.

The model is tested for prediction and test results are defined as y_prediction_1.

```
[ ] from sklearn.metrics import accuracy_score
    print(accuracy_score(y_test,y_prediction_1))

0.9033333333333333
```

Figure 27: Accuracy Score calculation.

The accuracy score metric is imported from the sklearn metric module, and the accuracy of the model is calculated.

Accuracy score = 90.33 percent.

```

from sklearn.metrics import confusion_matrix
cf_matrix = confusion_matrix(y_test, y_prediction_1)
sns.heatmap(cf_matrix, annot=True)
plt.title("Confusion Matrix for Logistic Regression", fontsize=14, fontname="DejaVu Sans", y=1.03)

```

Figure 28: Confusion metrics command

The confusion metrics is imported from the sklearn metric module in the sklearn library and is plotted as heatmap using the seaborn library. The plot title, font size, name and width is also defined.

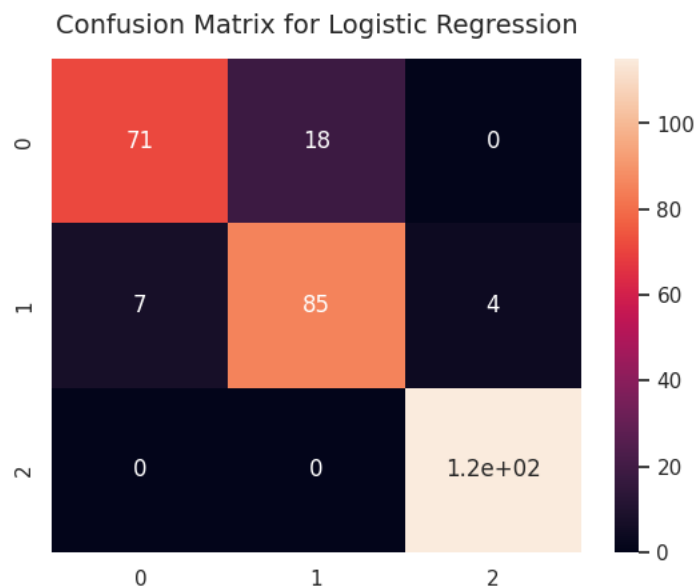


Figure 29: Confusion Matrix for Logistic Regression

The confusion metrics for three classes, shows the True Positives, False Negatives, False Positives and True Negatives for each of the classes.

```

from sklearn.metrics import classification_report
print(classification_report(y_test,y_prediction_1))

```

	precision	recall	f1-score	support
1	0.91	0.80	0.85	89
2	0.83	0.89	0.85	96
3	0.97	1.00	0.98	115
accuracy			0.90	300
macro avg	0.90	0.89	0.90	300
weighted avg	0.90	0.90	0.90	300

Figure 30: Classification Report for Logistic Regression

The classification report is another metrics for performance evaluation and is imported from sklearn metrics module. This is used to predict the accuracy, precision, recall and F1 score of the model.

Accuracy = 90 percent

Maximum Precision = 97 percent

Maximum recall = 100 percent

Maximum F1 score = 98 percent

```
from sklearn.neighbors import KNeighborsClassifier
```

Figure 31: Importing K Nearest Neighbors.

In this step, the K Nearest Neighbors classifier, which is present in the sklearn' s neighbors module within the sklearn library is imported.

```
[ ] model_2 = KNeighborsClassifier(n_neighbors=13,p=2,metric='euclidean')
      model_2.fit(X_train,y_train)
```

▼

KNeighborsClassifier

KNeighborsClassifier(metric='euclidean', n_neighbors=13)

Figure 32: Training the model.

The K-Nearest Neighbors classifier model is defined as model_2 and the Neighbors to be considered are set as 13 and the Euclidean distance, which is default for KNN is used as distance metric.

```
[ ] y_prediction_2= model_2.predict(X_test)
```

Figure 33: Testing the model.

The model is tested for prediction and test results are defined as y_prediction_2.

```
[ ] print(accuracy_score(y_test,y_prediction_2))
```

0.9366666666666666

Figure 34: Accuracy Score calculation.

The accuracy score metric imported from the sklearn metric module is used for calculating the accuracy of the model.

Accuracy score = 93.67 percent.

```
[ ] print(classification_report(y_test,y_prediction_2))
```

	precision	recall	f1-score	support
1	0.86	0.99	0.92	89
2	0.99	0.81	0.89	96
3	0.97	1.00	0.98	115
accuracy			0.94	300
macro avg	0.94	0.93	0.93	300
weighted avg	0.94	0.94	0.94	300

Figure 35: Classification Report for Logistic Regression

The classification report is another metrics for performance evaluation and is imported from sklearn metrics module. This is used to predict the accuracy, precision, recall and F1 score of the model.

Accuracy = 94 percent

Maximum Precision = 99 percent

Maximum recall = 100 percent

Maximum F1 score = 98 percent

```
from sklearn.metrics import confusion_matrix
cf_matrix = confusion_matrix(y_test, y_prediction_2)
sns.heatmap(cf_matrix, annot=True)
plt.title("Confusion Matrix for KNeighborsClassifier", fontsize=14, fontname="DejaVu Sans", y=1.03)
```

Figure 36: Confusion metrics command

The confusion metrics is imported from the sklearn metric module in the sklearn library and is plotted as heatmap using the seaborn library. The plot title, font size, name and width is also defined.

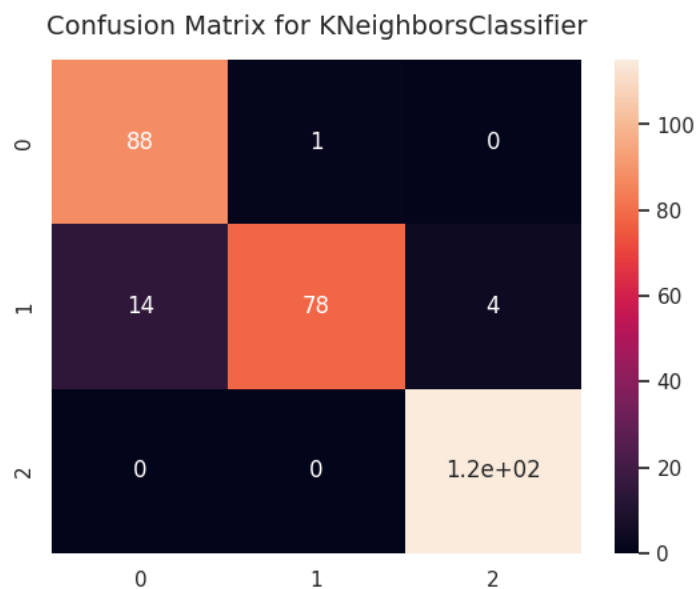


Figure 37: Confusion Matrix K-Nearest Neighbors.

The confusion metrics for three classes, shows the True Positives, False Negatives, False Positives and True Negatives for each of the classes.

```
[ ] from sklearn.ensemble import GradientBoostingClassifier
```

Figure 38: Importing Gradient Boosting Classifier.

In this step, the Gradient Boosting Classifier, which is present in the sklearn' s ensemble module, within the sklearn library is imported.

```
[ ] model_3 = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
)
```

```
[ ] model_3.fit(X_train,y_train)
```

```
▼ GradientBoostingClassifier
GradientBoostingClassifier()
```

Figure 39: Training the model.

The Gradient Boosting classifier model is defined as model_3 and since the Gradient Boosting is an ensemble model, the base or weak learners were defined as 100, learning rate as 0.1 and maximum depth as 3.

```
▶ y_prediction_3= model_3.predict(X_test)
```

Figure 40: Testing the model.

The model is tested for prediction and test results are defined as y_prediction_3.

```
[ ] print(accuracy_score(y_test,y_prediction_3))  
  
1.0
```

Figure 41: Accuracy Score calculation.

The accuracy score metric imported from the sklearn metric module is used for calculating the accuracy of the model.

Accuracy score = 100 percent.

```
▶ print(classification_report(y_test,y_prediction_3))
```

	precision	recall	f1-score	support
1	1.00	1.00	1.00	89
2	1.00	1.00	1.00	96
3	1.00	1.00	1.00	115
accuracy			1.00	300
macro avg	1.00	1.00	1.00	300
weighted avg	1.00	1.00	1.00	300

Figure 42: Classification Report for Gradient Boosting Classifier

The classification report is another metrics for performance evaluation and is imported from sklearn metrics module. This is used to predict the accuracy, precision, recall and F1 score of the model.

Accuracy = 100 percent

Maximum Precision = 100 percent

Maximum recall = 100 percent

Maximum F1 score = 100 percent

```
[ ] from sklearn.metrics import confusion_matrix
    cf_matrix = confusion_matrix(y_test, y_prediction_3)
    sns.heatmap(cf_matrix, annot=True)
    plt.title("Confusion Matrix for GradientBoostingClassifier", fontsize=14, fontname="DejaVu Sans", y=1.03)
```

Figure 43: Confusion metrics command

The confusion metrics is imported from the sklearn metric module in the sklearn library and is plotted as heatmap using the seaborn library. The plot title, font size, name and width is also defined.

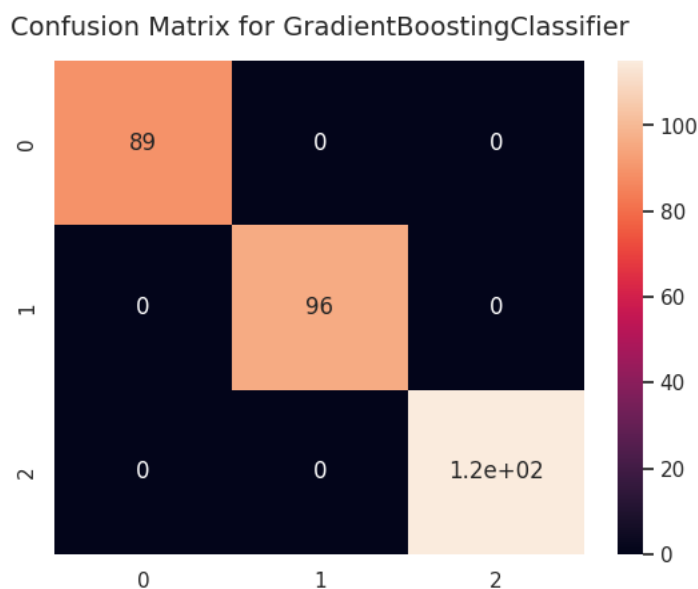


Figure 44: Confusion Matrix for Gradient Boosting Classifier

The confusion metrics for three classes, shows the True Positives, False Negatives, False Positives and True Negatives for each of the classes.

```
[ ] from sklearn.tree import DecisionTreeClassifier
```

Figure 45: Importing Decision Tree Classifier.

In this step, the Decision Tree Classifier, which is present in the sklearn's tree module within the sklearn library is imported.

```
[ ] model_4= DecisionTreeClassifier(max_depth=None)
```

```
[ ] model_4.fit(X_train,y_train)
```

```
▼ DecisionTreeClassifier  
DecisionTreeClassifier()
```

Figure 46: Training the model.

The Decision Tree Classifier is defined as model_4 and maximum depth for the tree is defined as none.

```
▶ y_prediction_4= model_4.predict(X_test)
```

Figure 47: Testing the model.

The model is tested for prediction and test results are defined as y_prediction_4.

```
[ ] print(accuracy_score(y_test,y_prediction_4))
```

```
1.0
```

Figure 48: Accuracy Score calculation.

The accuracy score metric imported from the sklearn metric module is used for calculating the accuracy of the model.

Accuracy score = 100 percent.

```
[ ] print(classification_report(y_test,y_prediction_4))
```

	precision	recall	f1-score	support
1	1.00	1.00	1.00	89
2	1.00	1.00	1.00	96
3	1.00	1.00	1.00	115
accuracy			1.00	300
macro avg	1.00	1.00	1.00	300
weighted avg	1.00	1.00	1.00	300

Figure 49: Classification report for Decision Tree

The classification report is another metrics for performance evaluation and is imported from sklearn metrics module. This is used to predict the accuracy, precision, recall and F1 score of the model.

Accuracy = 100 percent

Maximum Precision = 100 percent

Maximum recall = 100 percent

Maximum F1 score = 100 percent

```
from sklearn.metrics import confusion_matrix
cf_matrix = confusion_matrix(y_test, y_prediction_4)
sns.heatmap(cf_matrix, annot=True)
plt.title("Confusion Matrix for DecisionTreeClassifier", fontsize=14, fontname="DejaVu Sans", y=1.03)
```

Figure 50: Confusion metrics command

The confusion metrics is imported from the sklearn metric module in the sklearn library and is plotted as heatmap using the seaborn library. The plot title, font size, name and width is also defined.

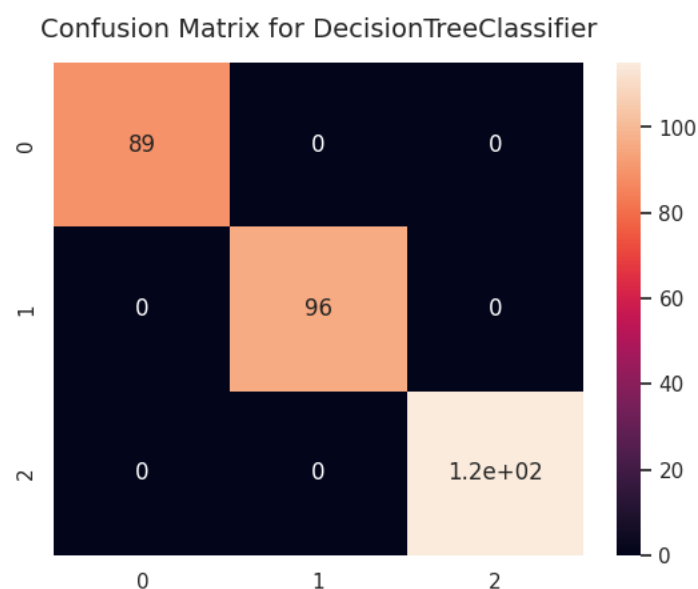


Figure 51: Confusion Matrix for Decision Tree

The confusion metrics for three classes, shows the True Positives, False Negatives, False Positives and True Negatives for each of the classes.

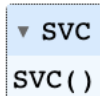
```
[ ] from sklearn.svm import SVC
```

Figure 52: Importing Support Vector Machine

In this step, the Support Vector Machine model, which is present in the sklearn's svm module within the sklearn library is imported.

```
[ ] model_5 = SVC(C=1.0, kernel='rbf', gamma='scale')
```

```
[ ] model_5.fit(X_train, y_train)
```



▼ SVC
SVC()

Figure 53: Training the model.

The Support Vector Machine model is defined as model_5 and for the Support Vector Machine, the C value, which is regularisation strength parameter is defined as 1, kernel is set as Radial Basis function kernel and decision boundary shape control parameter gamma as scale.



```
y_predictions_5 = model_5.predict(X_test)
```

Figure 54: Testing the model.

The model is tested for prediction and test results are defined as y_prediction_5.

```
[ ] print(accuracy_score(y_test, y_predictions_5))
```

```
0.9533333333333334
```

Figure 55: Accuracy Score calculation.

The accuracy score metric imported from the sklearn metric module is used for calculating the accuracy of the model.

Accuracy score = 95.33 percent.

```
print(classification_report(y_test,y_predictions_5))
```

	precision	recall	f1-score	support
1	1.00	0.89	0.94	89
2	0.93	0.96	0.94	96
3	0.94	1.00	0.97	115
accuracy			0.95	300
macro avg	0.96	0.95	0.95	300
weighted avg	0.96	0.95	0.95	300

Figure 56: Classification Report for Support Vector Machine

Classification report is another metrics for performance evaluation and is imported from sklearn metrics module. This is used to predict the accuracy, precision, recall and F1 score of the model.

Accuracy = 95 percent

Maximum Precision = 100 percent

Maximum recall = 100 percent

Maximum F1 score = 97 percent

```
from sklearn.metrics import confusion_matrix
cf_matrix = confusion_matrix(y_test, y_predictions_5)
sns.heatmap(cf_matrix, annot=True)
plt.title("Confusion Matrix for Support Vector Classifier", fontsize=14, fontname="DejaVu Sans", y=1.03)
```

Figure 57: Confusion metrics command

The confusion metrics is imported from the sklearn metric module in the sklearn library and is plotted as heatmap using the seaborn library. The plot title, font size, name and width is also defined.

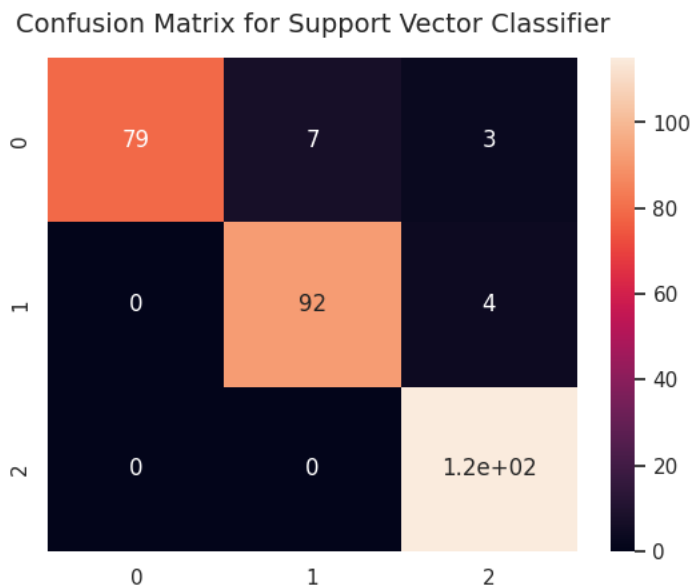


Figure 58: Confusion Matrix for Support Vector Machine

The confusion metrics for three classes, shows the True Positives, False Negatives, False Positives and True Negatives for each of the classes.

```
from sklearn.model_selection import cross_val_score

# List of classifiers
classifiers = [
    DecisionTreeClassifier(),
    SVC(),
    LogisticRegression(),
    GradientBoostingClassifier(),
    KNeighborsClassifier()
]

# Perform 5-fold cross-validation for each classifier
for classifier in classifiers:
    scores = cross_val_score(classifier, X, y, cv=5) # 5-fold cross-validation
    avg_score = sum(scores) / len(scores)
    print(f"{classifier.__class__.__name__}: {scores} - Average Score: {avg_score:.3f}")
```

Figure 59: Cross Validation

Since multiple models have shown higher accuracy values, a cross validation was done to evaluate the performance. Here, 5-fold cross validation was imported from the sklearn model selection module and the classifiers were selected and cross validation is conducted, by defining the 5-fold cross validation model.

```

DecisionTreeClassifier: [1. 1. 1. 1. 1.] - Average Score: 1.000
SVC: [0.945 0.94 0.96 0.95 0.95] - Average Score: 0.949
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:458: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
n_iter_i = _check_optimize_result(
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:458: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
n_iter_i = _check_optimize_result(
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:458: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
n_iter_i = _check_optimize_result(
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:458: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
n_iter_i = _check_optimize_result(
LogisticRegression: [0.925 0.915 0.96 0.96 0.955] - Average Score: 0.943
GradientBoostingClassifier: [1. 1. 1. 1. 1.] - Average Score: 1.000
KNeighborsClassifier: [0.995 1. 0.995 1. 1. ] - Average Score: 0.998

```

Figure 60: 5-fold Cross-validation results.

The results showed that, the models Decision Tree Classifier and Gradient Boosting had an average accuracy of 100 percent, whereas KNN, Logistic regression and SVC showed an accuracy of 99.8 percent, 94.3 percent and 94.9 percent respectively.

CHAPTER 5

CONCLUSION

5.1 OVERVIEW

This chapter will summarise the findings of the research and the answer to the research questions, hence, provide conclusions on the achievements of the research aim and objectives.

5.2. CONCLUSION

The research was conducted in the scenario of increasing cases of lung cancer among people of different age groups and this being a life-threatening disease with very severe complications as symptoms, it is essential to predict and increase the chance of detection at early stages or complete prevention of the disease. The main objective of this study was to create a machine-learning model that can accurately forecast lung cancer cases using a variety of datasets. The proposed model should be able to perform better in terms of lung cancer stage and early detection and also to highlight the most significant risk factors associated with the development of lung cancer as finding these components can boost efforts at prevention and aid in understanding the disease's underlying causes. So, in this research, based on the basic research conducted, the best models were identified and trained. The models were used for prediction and their prediction accuracy and overall performance was evaluated. The results of the evaluation are as follows:

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	90.33	97	100	98
K-Nearest Neighbors	93.67	99	100	98
Gradient Boosting	100	100	100	100
Decision Tree	100	100	100	100
Support Vector Machine	95.33	100	100	97

This result shows that the Gradient Boosting Classifier and Decision Tree Classifier outperformed other models in terms of all evaluation metrics. But these models are cross validated using 5-fold cross validation and the results showed that Gradient Boosting Classifier and Decision Tree Classifier once again outperformed other models with an average accuracy of 100 percent, whereas KNN, Logistic regression and SVC showed an accuracy of 99.8 percent, 94.3 percent and 94.9 percent respectively. Hence, these models can be used for further development of systems for early detection and prediction of lung cancer in individuals.

The analysis has also identified the most important features for the prediction of lung cancer, which are Coughing of Blood, Dust Allergy, Passive Smoker, Occupational Hazards, Air Pollution, chronic Lung Disease, Shortness of Breath, Dry Cough, Snoring, and Swallowing Difficulty. The patients are to be mindful of these and can be a sign of lung cancer and help in early detection.

REFERENCES

Patra, R. (2020) "Prediction of lung cancer using machine learning classifier," in *Communications in Computer and Information Science*. Singapore: Springer Singapore, pp. 132–142.

Faisal, M. I. *et al.* (2018) "An evaluation of machine learning classifiers and ensembles for early stage prediction of lung cancer," in *2018 3rd International Conference on Emerging Trends in Engineering, Sciences and Technology (ICEEST)*. IEEE.

Faezeh Hosseinzadeh, Amir Hossein KayvanJoo, Mansuor Ebrahimi & Bahram Goliaei (2013) "Prediction of lung tumor types based on protein attributes by machine learning algorithms," *springerplus*.

Anil Kumar, C., Harish, S., Ravi, P., Svn, M., Kumar, B.P., Mohanavel, V., Alyami, N.M., Priya, S.S. and Asfaw, A.K., 2022. Lung cancer prediction from text datasets using machine learning. *BioMed Research International*, 2022.

Ma, B., Meng, F., Yan, G., Yan, H., Chai, B. and Song, F., 2020. Diagnostic classification of cancers using extreme gradient boosting algorithm and multi-omics data. *Computers in biology and medicine*, 121, p.103761.

Peter, S., Diego, F., Hamprecht, F.A. and Nadler, B., 2017. Cost efficient gradient boosting. *Advances in neural information processing systems*, 30.

Punmiya, R. and Choe, S., 2019. Energy theft detection using gradient boosting theft detector with feature engineering-based preprocessing. *IEEE Transactions on Smart Grid*, 10(2), pp.2326-2329.

Priyam, A., Abhijeeta, G.R., Rathee, A. and Srivastava, S., 2013. Comparative analysis of decision tree classification algorithms. *International Journal of current engineering and technology*, 3(2), pp.334-337.

Myles, A.J., Feudale, R.N., Liu, Y., Woody, N.A. and Brown, S.D., 2004. An introduction to decision tree modeling. *Journal of Chemometrics: A Journal of the Chemometrics Society*, 18(6), pp.275-285.

Kingsford, C. and Salzberg, S.L., 2008. What are decision trees?. *Nature biotechnology*, 26(9), pp.1011-1013.

Jadhav, S.D. and Channe, H.P., 2016. Comparative study of K-NN, naive Bayes and decision tree classification techniques. *International Journal of Science and Research (IJSR)*, 5(1), pp.1842-1845.

Zhang, S., Li, X., Zong, M., Zhu, X. and Cheng, D., 2017. Learning k for knn classification. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 8(3), pp.1-19.

Uddin, S., Haque, I., Lu, H., Moni, M.A. and Gide, E., 2022. Comparative performance analysis of K-nearest neighbour (KNN) algorithm and its different variants for disease prediction. *Scientific Reports*, 12(1), p.6256.

Zou, X., Hu, Y., Tian, Z. and Shen, K., 2019, October. Logistic regression model optimization and case analysis. In *2019 IEEE 7th international conference on computer science and network technology (ICCSNT)* (pp. 135-139). IEEE.

Lemon, S.C., Roy, J., Clark, M.A., Friedmann, P.D. and Rakowski, W., 2003. Classification and regression tree analysis in public health: methodological review and comparison with logistic regression. *Annals of behavioral medicine*, 26, pp.172-181.

